THILINA PERERA

S25021960_M.A.A.T.Perera_assignment 2_COM754 - TN

Turnitin Similarity Checker - check your own work

Document Details

Submission ID
trn:oid:::2945:340384232

Submission Date
Jan 8, 2026, 8:42 PM GMT+5:30

Download Date
Jan 8, 2026, 8:45 PM GMT+5:30

File Name
S25021960_M.A.A.T.Perera_assignment 2_COM754 - TN.docx

File Size
209.1 KB

12 Pages

2,749 Words

16,432 Characters

1% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography

Match Groups

2 Not Cited or Quoted 1%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

0% Internet sources
0% Publications
0% Submitted works (Student Papers)

Match Groups

- **2** Not Cited or Quoted 1%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

0% Internet sources
0% Publications
0% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1 Student papers

Surigao State College of Technology - Engineering Department on 2026-01-06 <1%

2 Internet

core.ac.uk <1%

The Influence of Developer Expertise on the Quality of AI-Assisted Code: A Small-Scale Study of React Front-End Application Development.

S25021960_M.A.A.T.Perera_assignment 2_COM754

1. Introduction

1.1. Background of the study

The inclusion of Large Language Models (LLMs) in the software development life cycle has caused an enormous shift in the way code is written. Surveys conducted currently report that LLMs support about 92% of developers in their day-to-day activities, for example, the generation of boiler-plate, suggestions for possible solutions to problems, assistance with refactoring and debugging, and summarizing of content [1]. Although the short term successes have been substantial, the long-term implications of this type of influence on software quality remain a serious concern [2].

1.2. Motivation and Rationale for the Research

Empirical studies show that even though there have been productivity increases from AI assisted coding, the generated code often contains critical vulnerabilities and structural flaws. A research study focused on open source code snippets reported that AI generated code contained CWE (Common Weakness Enumeration) security weaknesses [3], categorized into forty-three CWEs (Common Weakness Enumerations). The three most dangerous CWEs were Cross-Site Scripting (CWE-79), inadequate randomness of values (CWE-330), and inadequate control over code generation (CWE-94) [3]. Additionally, AI generated code is also found to be less complex and less variable, but is much more likely to exhibit "Implementation Laziness," i.e. developers will leave out complex business logic to meet the token limit [4] [5]

1.3. Research questions and objectives

Although AI tools can produce working results, the quality of the finished software depends heavily on the developer's ability to identify the erosion of architecture and technical debt when developing AI-assisted code [5]. Quality of high-level code has five characteristics: validity, correctness, security, reliability, and maintainability [2]. The purpose of this study is to perform an exploratory analysis of a limited scope to understand how different levels of developer experience affects the final quality of AI-assisted code with regard to those five qualities.

Research Questions

- RQ1. How does developer expertise affect the overall quality of AI-assisted React code for a controlled feature task?
- RQ2. Which AI-code quality issues (e.g., over-engineering, dependency problems, and testing gaps) vary most by expertise?

Research Objectives

- Compare the quality of AI-assisted React code across developer expertise levels.

- Identify which quality factors are most affected by expertise (maintainability, dependencies, testing/error handling).

1.4. Scope and Significance of the Study

This descriptive research paper tests the React front-end development by performing a low-quality baseline code within an Item Master module with the help of AI-oriented remediation of the baseline code with Visual studio code and GitHub Copilot. The measurement of outcomes is performed through SonarQube, which concentrates on quality measurements, such as the number of issues, their duplication and complexity, as well as test coverage, and evaluates task completion measures. The research paper will uncover the relationship between expert knowledge of the developers and quality improvement of the code in a controlled setting. It provides useful information on the connection between the seniority of developers and the achievements with the assistance of AI, offering evidence-based [6] [7]

2. Literature Review

2.1. Key Concepts and Prior Work

AI code assistants have changed how developers use their tools in Integrated Development Environments (in-IDE HAX) [8]. This area is concerned with "functional correctness," which means the code passes tests, and "structural quality," which means how well the code follows design patterns. Research indicates that while tools like GitHub Copilot can speed up tasks by 55%, the resulting code quality may be unpredictable [2] [9]. Over 80 studies show that AI increases productivity but requires developers to verify and fix code [8].

Recent benchmarks like GSO-bench show that even cutting-edge LLMs struggle with “Generative Software Operations,” often losing architectural integrity when tasked with complex remediation rather than simple generation [10]. Empirical studies on 500k code samples show that AI code is simpler and more repetitive than human work, resulting in higher security vulnerabilities (up to 8% in some models) than human baselines [4].

2.2. Challenges and Ethical Issues

The main technical problem is the “Asleep at the Keyboard” effect, which happens when developers rely too much on AI suggestions [11]. This leads to ethical concerns regarding accountability for insecure codes merged into production. Moreover, “implementation laziness” occurs when models satisfy token constraints by omitting critical logic, effectively pushing technical debt onto the human developer [4], [5]. In React development, this often shows up as issues with component lifecycles breaking or state management being inefficient - problems that automated tools might not always catch or label as “errors.”

2.3. Gap in Existing Knowledge

A lot of recent research has been about “Greenfield” development, which means starting from scratch and writing new code using simple prompts. There is a significant gap in research regarding “Brownfield” remediation, where developers must use AI to fix or improve *existing* complex React codebases. Crucially, while some studies touch on expertise [12], there is a lack of granular data on how developer seniority specifically influences the quality of remediated front-end code under high-pressure, fixed-time constraints. This study addresses this by measuring the mediator role of expertise in a controlled React feature task.

2.4. Theoretical Framework and Measurement

This research utilizes the Grounded Copilot Model [9], which categorizes developer-AI interaction into “Exploration” and “Acceleration” phases. Expertise is hypothesized to shift a developer's behavior from simple acceleration (blind acceptance) to critical exploration (refinement).

This research uses SonarQube to evaluate the quality of the generated code. SonarQube is a full-featured platform to manage code quality and it functions as an additional "validation" layer in that it can identify bugs, security vulnerabilities and code smells that are introduced into the generated code by AI-based models [6]. This allows us to provide standardized metrics (Technical Debt Ratio and Maintainability Index) for comparing the novice authored code with expert authored code, providing a repeatable and data-driven comparison.

3. Methodology

3.1. Research Approach and Justification

This study uses an exploratory quantitative approach to determine how developer experience affects AI-assisted code fix solutions for React front-end tasks. This project should use a quantitative method because the results will have numbers that can be compared (such as issue counts, coverage, duplication, and complexity) [7]. SonarQube quantifies reliability, security, maintainability, coverage, and duplication issues so we can objectively evaluate the code before and after the task.

Since this is a small-scale study, our goal is not statistical generalizability, but to determine how to improve code quality within time constraints.

3.2. Research Method

The research method is a controlled code remediation experiment. All participants receive the same baseline React “Item Master” codebase and are asked to improve it within a fixed 20-minute window using VS Code and GitHub Copilot (Claude Opus 4.5). The baseline project is analyzed in SonarQube prior to the task, and each participant’s final submission is analyzed again using the same SonarQube configuration.

This method was selected because it

- Controls the starting point (same codebase for all participants),
- Applies the same time pressure to all participants,
- Produces outputs that can be measured consistently using SonarQube metrics (bugs, vulnerabilities, code smells, coverage, duplication, hotspots, etc.).

3.3. Research Design

A pre-post design was used:

- Pre (T0): Baseline SonarQube scan of the initial codebase
- Post (T1): SonarQube scan of each participant's final submission after 20 minutes

Independent Variables

Developer expertise level, measured and categorized using a weighted questionnaire (see Appendix A)

Dependent Variables

Primary quantitative outcomes extracted from SonarQube:

- Code smells
- Bugs
- Vulnerabilities
- Security hotspots
- Test coverage
- Code duplication
- Quality Gate status

Derived Measures (Quality Uplift)

Improvement was calculated as the difference between baseline and final values:

- Δ Smells
- Δ Bugs
- Δ Vulnerabilities
- Δ Coverage

3.4. Participants and Sampling

Participants were recruited using convenience sampling from the researcher's academic and professional network. Inclusion criteria:

- Age 18 or above
- Ability to run a React project
- Basic familiarity with GitHub workflows

3.5. Instruments and Materials

3.5.1. Developer Expertise Questionnaire

Developer expertise was operationalized using a structured Google Form combining:

- Self-assessment of React and AI-assisted coding experience
- Years of experience and role level
- Multiple-choice knowledge questions on React fundamentals and AI-assisted coding practices

Scores were aggregated into a 0-100 scale to provide a more robust classification than relying on a single indicator.

3.5.2. Codebase and Task Pack

Participants received:

- A GitHub repository containing the baseline “Item Master” React project
- A task brief outlining objectives, constraints, and submission requirements
- A summary of the baseline SonarQube report to guide remediation priorities

3.5.3. Secondary Process Measures

To support interpretation of results, the following were recorded:

- Time spent (fixed at 20 minutes; deviations noted)
- Number of AI prompts used and brief prompt logs
- Short participant notes describing major changes made

3.6. Procedure

Pre-Task: Participants completed a questionnaire that categorized participants into a group based on their self-reported level of expertise in software development.

Setup: Each participant cloned the repository and then restored the dependency chain by running npm install or yarn install (as appropriate).

Remediation Task: Participants could use AI-assisted tools for up to 20 minutes to remediate issues identified within the SonarQube report. Once they finished, each participant would submit the updated code with accompanying notes/logs from prompt activity.

Analysis Post-Task: The researcher performed SonarQube scans on all submitted projects using the same configuration used when performing the baseline scan to compare the results.

3.7. Reliability, Validity, and Controls

Internal validity was supported through:

- A single baseline codebase,
- A fixed time limit,
- Consistent tooling and SonarQube configuration for all participants.

3.8. Ethical Considerations

The participant's rights to confidentiality were protected by ensuring participants' anonymity, providing them with a code (e.g. P01-P05), obtaining consent electronically as well as giving participants an opportunity to opt out of this research at any time. Anonymity, and therefore participant rights, were further protected by storing all data on secure servers that could only be accessed by a limited number of individuals; all data collected was also presented in a way that made it impossible for participants to identify themselves (i.e., aggregate reporting).

4. Results and Findings

4.1. Overview of Collected Data

Data were collected from two sources:

1. Developer expertise questionnaire (Appendix B)
2. SonarQube static analysis of the baseline and final submissions (Appendix C & D)

4.2. Participant Expertise Classification

All **five participants (n=5)** completed the expertise identification questionnaire (Appendix A) and met the eligibility requirement (**18+**). Based on the computed and the defined thresholds (in section 3.3), participants were grouped as **Novice (2/5)**, **Intermediate (2/5)**, and **Advanced (1/5)**.row figures are provided in Appendix B.

S25021960_M.A.A.T.Perera_assignment 2_COM754

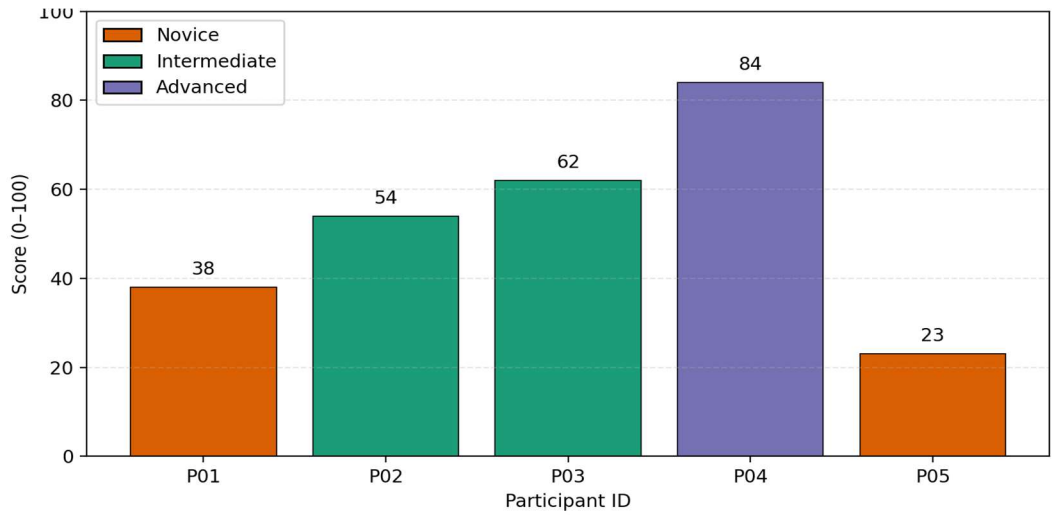

Figure 1: Expertise classification scores by participant

4.3. Quantitative Results (Baseline vs Post Task)

4.3.1. Baseline Code Quality

The initial codebase was analyzed using SonarQube to establish a reference point for later comparison. Key baseline metrics As bellow (Screen attached in Appendix C):

- Quality Gate : failed
- Lines of Code (LOC): 629
- Bugs: 1
- Vulnerabilities: 0
- Code Smells: 45
- Security Hotspots: 1
- Coverage: 0.0%
- Duplications: 0.0%

4.3.2. Post task SonarQube Outcomes

Post remediation SonarQube dashboards were captured (Appendix D) and the corresponding metrics are reported in Table 1. These metrics are used as quantitative indicators of maintainability, security, reliability, duplication, and coverage.

Participant	P01(Novice)	P02(Intermediate)	P03(Intermediate)	P04(Advanced)	P05(Novice)
Quality Gate	Failed	Failed	Failed	Passed	Failed
Δ Lines of Code	97	65	-59	-103	-90
Δ Bugs	-1	-1	-1	-1	1
Δ Vulnerabilities	0	0	0	0	0
Δ Code Smells	-28	-33	-38	-45	-29
Δ Security Hotspots	-1	-1	-1	-1	0
Δ Coverage	0	0	43.1	84.7	0
Δ Duplications	0	0	0	0	0

Table 1: Delta (Δ) improvements in SonarQube metrics after 20-minute AI-assisted remediation.

S25021960_M.A.A.T.Perera_assignment 2_COM754

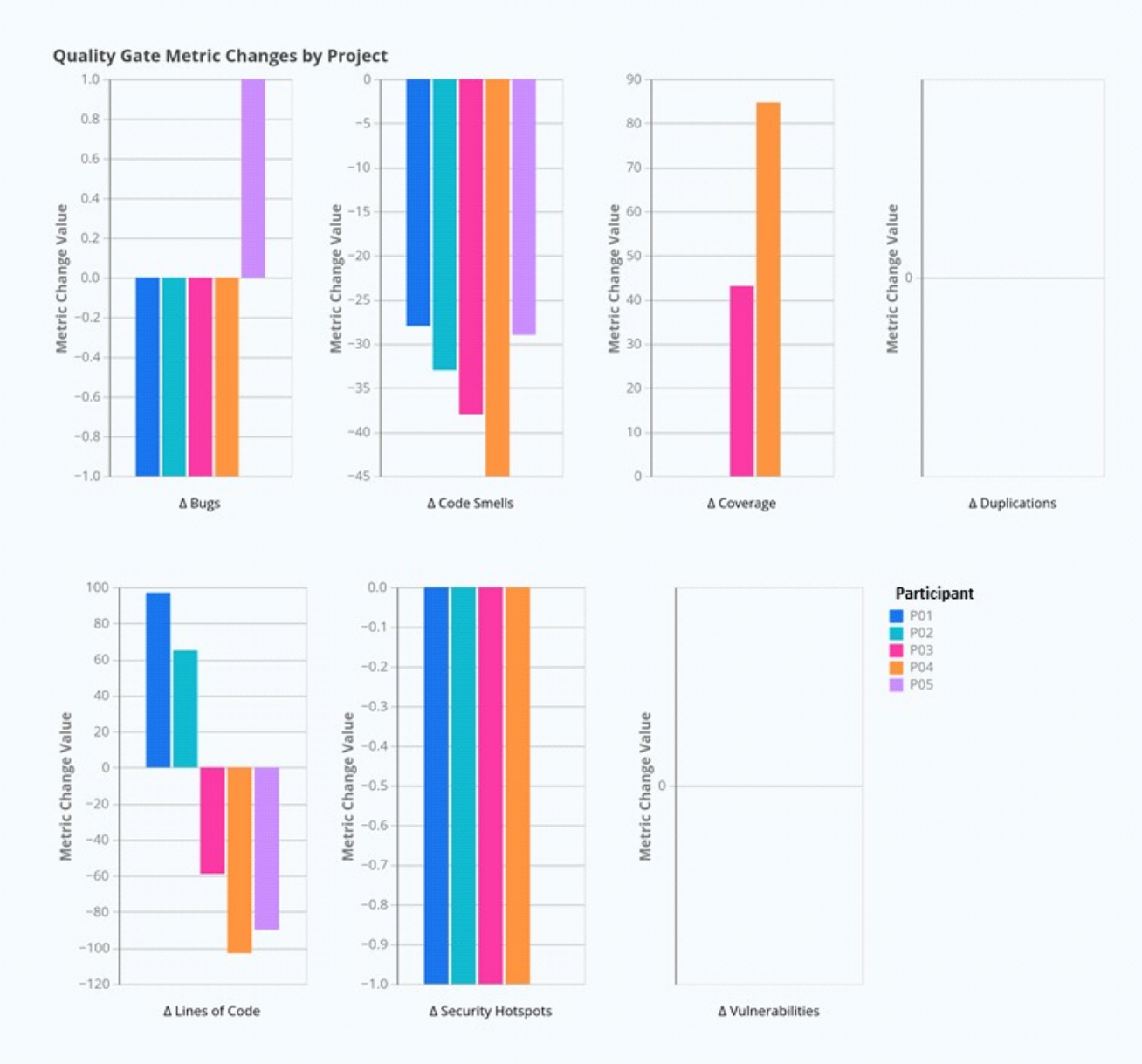

Figure 2: Delta (Δ) improvements in SonarQube metrics after 20-minute AI-assisted remediation.

4.4. Summary of Key Quantitative Findings

Some key patterns stood out from the results:

- **Expert Efficiency:** Advanced developers were much better at reducing technical debt, such as code smells and duplication, compared to the novices.

- **Novice Challenges:** One novice participant (P05) actually introduced new bugs while trying to fix issues, suggesting they may have relied too heavily on AI suggestions without double-checking them.
- **Managing Complexity:** Advanced developers made a clear effort to lower cognitive complexity, while novices either increased it or left it unchanged.

5. Discussion / Critical Analysis

5.1. Interpretation of Results

It is evident that the quality of AI-assisted code is strongly determined by the level of expertise of a developer. The Expert Efficiency in the case of RQ1 is equivalent to the Acceleration Phase in the Grounded Copilot Model [9], where the skilled developers apply AI to handle routine work, and instead concentrate on the overall architecture. The Novice Regressions (P05), in turn, point to one of the frequent problems: Automation Bias [8], in which inexperienced users overtrust the AI recommendations rather than their own judgment, and as a result, they combine code that works but has secret bugs in it.

5.2. Comparison with Literature

Our discovery that novices create more vulnerabilities is consistent with Perry et al. [12], who posited that experience negatively correlates with uncritical trust in AI. Furthermore, the "implementation laziness" discussed by Slater [5] was evident in the novice logs; P05 accepted a simplified version of a complex validation logic suggested by the AI, which SonarQube subsequently flagged as a "critical bug" due to missing edge-case handling. This observation confirms that AI tools frequently prioritize "token efficiency" over "logical completeness," a nuance that only expert developers seem to catch.

5.3. Implications

The practical implication is that **SonarQube Quality Gates** [6] are non-negotiable in AI-accelerated workflows, especially for junior teams. Academically, this study suggests that "productivity" cannot be measured by speed alone (LOC/hour) but must include "Verification Overhead" [8] the time an expert spends fixing the "lazy" mistakes of the AI.

5.4. Challenges & Limitations

The significant limitation of this research is that the sample size is quite small (n=5), and it is not easy to make general statistical conclusions. Also, convenience sampling can have led to certain bias, as all the participants belonged to the same professional network. The time constraint of 20 minutes could also have been disadvantageous to the novices since they usually require additional time to complete the overhead of verification. Finally, using SonarQube as the only tool to measure the quality of code might overlook some crucial details such as the intent of the developer or a certain best practice which are not easily identified by the static analysis tools.

6. Conclusion and Recommendations

6.1. Summary of key outcomes

This study investigates the impact of developer expertise on the quality of AI-assisted React code for a specific feature task. Utilizing SonarQube for quantitative analysis and issue categorization across various dimensions, the results indicate that while AI assistance can speed up code delivery, the quality of the output is contingent on the developers' ability to overview AI-generated suggestions.

6.2. Answering the Research Questions

RQ1: All developers improved overall quality after remediation (Table 1), but higher expertise produced greater improvements in all aspects.

RQ2: The issues that varied most by expertise were especially in test coverage (**testing gaps** and **maintainability-related decisions**) supporting more verifiable and maintainable code for future developers.

6.3. Recommendations

This study highlights that the quality of AI-assisted code really depends on the developer's experience, especially their understanding of architectural patterns, best practices, and maintainability. Because of this, teams should focus on ongoing training in React and software design for all developers. It's also important to give junior developers clear guidelines on how to use AI effectively, including helpful prompting tips and a simple review checklist. Lastly, adding automated CI/CD quality checks - particularly those that focus on new code through SonarQube quality gates - can help prevent quality issues by making sure all changes meet the required standards before they're merged.

6.4. Future work

Future work should extend this study by replicating the remediation task with a larger and diverse sample of developers, which improve external validity and allow stronger comparison. In addition to the quantitative outcomes, should incorporate qualitative interviews to explain why particular groups prioritize specific fixes under time pressure (e.g., refactoring vs. testing) and to capture decision-making factors such as confidence, risk perception, and review habits. Collecting and analyzing prompt logs (prompt content, iteration count, context provided, and verification steps) would enable deeper understanding of how "AI usage skill" interacts with developer expertise and influences quality uplift. Finally, the same methodology should be evaluated across other programming languages and ecosystems (e.g., backend services in Java/C# or scripting in Python), as AI-assisted code quality dynamics and static analysis signals may differ by language, tooling, and typical architectural patterns; repeating the study across stacks would clarify which findings are React-specific versus broadly generalizable.